Spec and test gaps: AON timer

Repository `targets/opentitan` at commit `b84154c71810`. Report generated 2026-11-11 01:08 UTC by chipbag.

Each finding below gives the gap, the evidence for it as exact quotes at their file and line, and how it was verified: an agent searched the repository to refute the finding, and the finding was kept only if that failed. Every quote was checked against the file at the cited lines. The report does not recommend a fix.

Summary

#	Category	Verified	Withdrawn	Inconclusive
1	Spec too ambiguous to test	3	7	0
2	Spec contradicts itself	1	1	0
3	Test contradicts its own description	3	4	0
4	Spec and test disagree	0	0	0
5	Spec not tested	17 (41 items)	9	0

Scope

Spec sources (4): `hw/ip/aon_timer/data/aon_timer.hjson`, `hw/ip/aon_timer/doc/interfaces.md`, `hw/ip/aon_timer/doc/programmers_guide.md`, `hw/ip/aon_timer/doc/theory_of_operation.md`

Verification sources (45 files): `hw/ip/aon_timer/dv/env/seq_lib/` (12), `hw/dv/tools/dvsim/testplans/` (6), `hw/ip/aon_timer/dv/env/` (5), `sw/device/tests/` (5), `hw/ip/aon_timer/dv/cov/` (4), `hw/ip/aon_timer/data/` (2), `hw/ip/aon_timer/dv/` (2), `hw/top_earlgrey/dv/` (2), `hw/ip/aon_timer/doc/` (1), `hw/ip/aon_timer/dv/sva/` (1), `hw/ip/aon_timer/dv/tests/` (1), `hw/top_earlgrey/data/` (1), `hw/top_earlgrey/data/ip/` (1), `hw/top_earlgrey/formal/conn_csvs/` (1), `sw/device/lib/dif/` (1). The full list is in the manifest.

144 requirements after merging duplicates; 90 need a test (the rest hold by construction). 74 tests (69 with source), 65 test plan items. 53 requirements are matched to a test.

Files the verifier used as evidence that are not in the manifest: `doc/contributing/hw/racl/README.md`, `hw/dv/sv/cip_lib/seq_lib/cip_base_vseq.sv`, `hw/dv/sv/tl_agent/tl_agent_cov.sv`, `hw/dv/tools/dvsim/tests/csr_tests.hjson`, `hw/ip/aon_timer/README.md`, `hw/ip/aon_timer/doc/registers.md`, `sw/device/tests/pwrmgr_sleep_all_wake_ups_impl.c`, `sw/device/tests/pwrmgr_smoketest.c`.

1. Spec too ambiguous to test

1.1 The policy selection vector parameter selects the RACL policy applied to each register.

`f_483ce24e65e4` · unclear referent · severity medium · section: hw/ip/aon_timer/data/aon_timer.hjson > aon_timer > inter_signal_list

Gap: The parameter is not named, and the selection encoding is not given.

Spec says:

- `hw/ip/aon_timer/data/aon_timer.hjson:116`: `The policy selection vector (parameter) selects the policy for each register.`
- `hw/ip/aon_timer/doc/interfaces.md:20`: `The policy selection vector (parameter) selects the policy for each register.`

Not answered by the spec: What is the name of the policy selection vector parameter (e.g. RaclPolicySelVec)? What is its width and encoding, i.e. one policy index per register in register-map order? What is its default value?

Verified: No specification text names the parameter or gives its width, encoding or default. Only the RTL does: RaclPolicySelVec, one entry per register, default all 0. The spec only repeats the vague sentence, in the hjson and in interfaces.md for many IPs.

Searched:

- RaclPolicySelVec in aon_timer in `hw/ip/aon_timer`: Only RTL files; no doc mentions it.
- 'policy selection vector' or RaclPolicySelVec in markdown docs in `hw/**/*.md`: Only the same boilerplate sentence in interfaces.md files; no definition.
- policy_sel terms in racl_ctrl docs in `hw/ip/racl_ctrl *.md`: No matches.

1.2 racl_error outputs the RACL error log information of this module.

`f_adb4f43d923a` · missing detail · severity medium · section: hw/ip/aon_timer/data/aon_timer.hjson > aon_timer > inter_signal_list

Gap: The text does not say when racl_error is asserted or what its content is.

Spec says:

- `hw/ip/aon_timer/data/aon_timer.hjson:126`: `RACL error log information of this module.`

Not answered by the spec: When is racl_error.valid asserted (on each RACL-denied read or write, for exactly one cycle)? Which racl_error_log fields are populated, and with what values (request address, write flag, role, ctn_uid, overflow)?

Verified: No spec text defines when aon_timer drives racl_error (valid timing or duration) or what values the fields carry. The racl_ctrl docs describe the consumer side: arbitration, overflow, clearing and the address register. They don't say how a subscribing IP drives its racl_error output.

Further evidence found by the verifier:

- `hw/ip/aon_timer/doc/interfaces.md:21`: `RACL error log information of this module.` — The aon_timer interface doc repeats the same one-line description and adds nothing.

Searched:

- racl_error in aon_timer docs and hjson in `hw/ip/aon_timer`: Only the one-line interface descriptions and a mention in theory_of_operation.md that the ports exist.

- RACL error log docs in `hw/**/*.md, including racl_ctrl theory_of_operation and registers`: Consumer-side arbitration, overflow and clearing behaviour. Nothing on the IP-side output timing or field values.

Looked at and ruled out:

- `hw/top_darjeeling/ip_autogen/racl_ctrl/doc/theory_of_operation.md:13-26`: Describes how racl_ctrl logs and clears errors, not when the aon_timer asserts racl_error or what its fields contain.

1.3 If the count remains above the threshold after clearing and the timer stays enabled, the wakeup event triggers again at the next clock tick.

`f_604ea5c51516` · unclear referent · severity medium · section: hw/ip/aon_timer/doc/programmers_guide.md > Programmer's Guide > Interrupt Handling

Gap: It is unclear which clock the 'next clock tick' refers to (AON clock or bus clock) and whether it is the prescaled tick.

Spec says:

- `hw/ip/aon_timer/doc/programmers_guide.md:93`

  > If {[`WKUP_COUNT_HI`](registers.md#wkup_count_hi), [`WKUP_COUNT_LO`](registers.md#wkup_count_lo)} remains above the threshold after clearing the interrupt or wakeup event and the timer remains enabled, the interrupt and wakeup event will trigger again at the next clock tick.

Not answered by the spec: After software clears INTR_STATE or WKUP_CAUSE, with the wakeup timer still enabled and the count at or above the threshold, when are the interrupt and wakeup request re-asserted? Is it the next AON clock cycle (comparison evaluated every AON cycle regardless of prescaler), or the next prescaler-derived count tick? What fixed synchronization latency should be expected before the bus-side INTR_STATE reflects it? Also, is the condition count >= threshold or count > threshold?

Verified: No other spec text says which clock "next clock tick" means, whether the prescaler applies, what the CDC latency is, or whether the comparison is >= or >. The theory of operation, the register descriptions and the hjson only say the threshold is "hit" or "reached".

Searched:

- prescaler, threshold, clock tick, >= in the aon_timer docs in `hw/ip/aon_timer/doc`: Only the programmers_guide line 93 describes re-trigger timing. Nothing defines the tick or the comparison operator.
- prescaler and threshold descriptions in `hw/ip/aon_timer/data/aon_timer.hjson`: Only generic descriptions such as 'hit the specified threshold'.
- read theory_of_operation.md in `hw/ip/aon_timer/doc`: Describes the CDC only in general terms, with no latency or tick definition.

Looked at and ruled out:

- `hw/ip/aon_timer/doc/registers.md:43`: It says each write resets the internal prescaler count, but it does not say when the comparison is evaluated or when the event re-asserts.

---

2. Spec contradicts itself

2.1 WKUP_THOLD_LO.threshold_lo occupies bits 31:0 and holds wakeup threshold bits [31:0].

`f_58017ac7abc1` · description vs. definition · severity high

Gap: The text names WKUP_THOLD_HI for bits[31:0], but the link target is wkup_thold_lo and HI already holds bits[63:32].

Spec says:

- `hw/ip/aon_timer/doc/theory_of_operation.md:32`: `Bits[31:0] in registers`
- `hw/ip/aon_timer/doc/theory_of_operation.md:32`: `WKUP_THOLD_HI`
- `hw/ip/aon_timer/data/aon_timer.hjson:218`: `bits: "31:0"`

Not answered by the spec: Is the register holding wakeup threshold bits[31:0] WKUP_THOLD_LO?

Verified: The theory_of_operation text lists WKUP_THOLD_HI twice. It names it for bits[63:32] and again for bits[31:0]. The second link's target is #wkup_thold_lo, so the link text is a typo. The hjson and RTL put bits 31:0 in WKUP_THOLD_LO, so the answer to the question is yes.

Searched:

- WKUP_THOLD definitions in the hjson and the doc text in `hw/ip/aon_timer`: WKUP_THOLD_HI holds bits 63:32 and WKUP_THOLD_LO holds bits 31:0. Only the doc link text is wrong.

---

3. Test contradicts its own description

3.1 aon_timer_wdog_lc_escalate_test: check cannot fail

`f_4377229b2488` · check cannot fail · severity medium

Gap: The plan (p1.k1) says the test fails in the interrupt handler if the bark interrupt fires. The comment i2 also says AON timer interrupts should not occur. The handler only calls LOG_ERROR on an unexpected AON timer interrupt and does not fail the test. A firing bark interrupt therefore cannot fail the test through this check, and the 'wdog disabled by escalation' claim goes unenforced there.

Labels in this finding: p1.k1 = `hw/top_earlgrey/data/ip/chip_aon_timer_testplan.hjson:122-123`; i2 = `sw/device/tests/aon_timer_wdog_lc_escalate_test.c:86`

Test description says:

- `hw/top_earlgrey/data/ip/chip_aon_timer_testplan.hjson:122-123`: `fail the test in the interrupt handler in case the bark interrupt is fired` — The test fails in the interrupt handler if the bark interrupt fires.
- `sw/device/tests/aon_timer_wdog_lc_escalate_test.c:86`: `We should not get aon timer interrupts since escalation suppresses them.`

Test does:

- `sw/device/tests/aon_timer_wdog_lc_escalate_test.c:89`: `LOG_ERROR("Unexpected aon timer interrupt %d", irq);` — check; does not fail the test; An AON timer interrupt is logged as an error but the handler does not fail the test itself.

Verified: The AON timer branch of ottf_handle_irq only calls LOG_ERROR and returns true. It uses no CHECK or fail macro, so a bark interrupt cannot fail the test there. The plan says the handler should fail the test. The alert-handler branch does use CHECK, but that is not the AON timer check. I did not read the rest of the test, so I did not look for another place that catches a bark.

Searched:

- Read the ISR in the test source in `sw/device/tests/aon_timer_wdog_lc_escalate_test.c:60-210`: The AON branch only logs. The alert-handler branch uses CHECK, which is a different check.

Looked at and ruled out:

- `sw/device/tests/aon_timer_wdog_lc_escalate_test.c:100-117`: These CHECKs are in the alert handler branch, not the AON timer branch.

3.2 aon_timer_prescaler: plan vs. test

`f_aefbd3a594cd` · plan vs. test · severity medium

Gap: Plan item p2 says the prescaler is changed while the timer is running. The test writes one random prescaler (s2) before the wakeup timer is enabled (s3) and never changes it again, so it never exercises a mid-run prescaler change.

Labels in this finding: p2 = `hw/ip/aon_timer/data/aon_timer_testplan.hjson`; s2 = `hw/ip/aon_timer/dv/env/seq_lib/aon_timer_prescaler_vseq.sv:57`; s3 = `hw/ip/aon_timer/dv/env/seq_lib/aon_timer_prescaler_vseq.sv:61`

Test description says:

- `hw/ip/aon_timer/data/aon_timer_testplan.hjson:44-45`: `WKUP_COUNT should reflect expected -and changed- passing of real time regardless of its changing prescaler value.` — WKUP_COUNT reflects the expected, changed passing of real time regardless of the changing prescaler value.

Test does:

- `hw/ip/aon_timer/dv/env/seq_lib/aon_timer_prescaler_vseq.sv:57`: `csr_utils_pkg::csr_wr(ral.wkup_ctrl.prescaler, prescaler);` — stimulus; Write random prescaler value to WKUP_CTRL.
- `hw/ip/aon_timer/dv/env/seq_lib/aon_timer_prescaler_vseq.sv:61`: `csr_utils_pkg::csr_wr(ral.wkup_ctrl.enable, 1'b1);` — stimulus; Enable the wakeup timer.

Verified: The plan item "prescaler" is explicitly linked to aon_timer_prescaler. It calls for switching the prescaler at random times while the timer runs. The vseq writes one random prescaler before enabling WKUP and never changes it again, so no mid-run change is exercised.

Further evidence found by the verifier:

- `hw/ip/aon_timer/data/aon_timer_testplan.hjson:37-41`: `Switch prescaler value at random times without locking configuration registers` — Plan stimulus is a prescaler change while running.

Searched:

- prescaler writes in the vseq and the base vseq in `aon_timer_prescaler_vseq.sv, aon_timer_base_vseq.sv`: Only one prescaler write, at line 57, before enable.
- plan item linkage in `aon_timer_testplan.hjson`: tests: ["aon_timer_prescaler"] explicitly

Looked at and ruled out:

- `hw/ip/aon_timer/data/aon_timer_testplan.hjson:51-71`: The jump plan item covers prescaler changes at random times, but it is a separate test and does not tie the prescaler plan item to a realizing test.

3.3 model_and_check_wdog_bark: comment vs. code

`f_fc10e42487be` · comment vs. code · severity low

Gap: The comment i2 says "Check reset_req pins", but the code that follows (k2/k3) checks the aon wakeup request pin (sample_pin idx 1) against predicted_wkup_req, not any reset request pin.

Labels in this finding: i2 = `hw/ip/aon_timer/dv/env/aon_timer_scoreboard.sv:1276`; k2 = `hw/ip/aon_timer/dv/env/aon_timer_scoreboard.sv:1282`; k3 = `hw/ip/aon_timer/dv/env/aon_timer_scoreboard.sv:1284`, `hw/ip/aon_timer/dv/env/aon_timer_scoreboard.sv:1287-1288`

Test description says:

- `hw/ip/aon_timer/dv/env/aon_timer_scoreboard.sv:1276`: `// Check reset_req pins:`

Test does:

- `hw/ip/aon_timer/dv/env/aon_timer_scoreboard.sv:1282`: `` `DV_CHECK_CASE_EQ(predicted_wkup_req, cfg.aon_intr_vif.sample_pin(.idx(1))) `` — check; When enabled and wkup_cause was not written in the same AON cycle, the predicted wkup_cause value must equal the aon wakeup request pin.
- `hw/ip/aon_timer/dv/env/aon_timer_scoreboard.sv:1284`: `else if (!(cfg.aon_intr_vif.sample_pin(.idx(1)) inside {0, predicted_wkup_req}))` — check; When enabled and wkup_cause was written in the same cycle, the aon wakeup request pin must be either 0 or the predicted value; otherwise a fatal is raised.
- `hw/ip/aon_timer/dv/env/aon_timer_scoreboard.sv:1287-1288` — check; When enabled and wkup_cause was written in the same cycle, the aon wakeup request pin must be either 0 or the predicted value; otherwise a fatal is raised.

  ```
  `uvm_fatal(`gfn, $sformatf("aon_wkup_req_o comparison failed (not matching 0/%0d)",
                                  predicted_wkup_req))
  ```

Verified: The comment says "Check reset_req pins:", but the code under it compares the aon wakeup request pin (sample_pin idx 1) with the predicted wkup_cause. It does not check any reset request pin. The mismatch is a stale comment and has no functional effect.

Searched:

- Context around the comment in model_and_check_wdog_bark in `aon_timer_scoreboard.sv lines 1235-1298`: The block checks only wkup_cause against sample_pin idx 1. No reset request pin is checked.

---

4. Spec and test disagree

No verified findings.

5. Spec not tested

5.1 Partly tested: 3 requirements in aon_timer.hjson > features (and 3 more sections)

`f_9207ee7a3ab2` · severity high

1. **Requirement** `cr_5c5513777815`: AON_TIMER.WATCHDOG.PAUSE: a paused watchdog resumes after wake-up.
   Section: hw/ip/aon_timer/data/aon_timer.hjson > aon_timer > features

   - `hw/ip/aon_timer/data/aon_timer.hjson:158`: `resuming after wake-up`

   Tested by `aon_timer_sleep_wdog_sleep_pause_test`**:** Configures the watchdog to pause in sleep, enters deep sleep, and after wake-up checks that the watchdog count stayed paused (smaller than the wakeup count).
   Not tested: Does not check that the watchdog actually resumes counting after wake-up; it only checks that the count was held during sleep.

   Verified: The search found no test that checks the watchdog resumes counting after wake-up. Only aon_timer_sleep_wdog_sleep_pause_test touches the item, and it checks that the count stayed paused.

2. **Requirement** `cr_346ab297701a`: Each write to WKUP_CTRL resets the internal prescaler count.
   Section: hw/ip/aon_timer/data/aon_timer.hjson > aon_timer > WKUP_CTRL

   - `hw/ip/aon_timer/data/aon_timer.hjson:183`: `Each write to the register resets the internal prescaler count.`

   Tested by `aon_timer_prescaler`**:** Writes WKUP_CTRL and then enables the wakeup timer, and the scoreboard checks the interrupt timing against a model that includes the prescaler delay. This only indirectly depends on the prescaler count being reset by a write.
   Not tested: No scenario writes WKUP_CTRL while the prescaler is mid-count, and nothing checks the prescaler count reset on that write.

   Verified: I found no scenario that writes WKUP_CTRL while the prescaler is mid-count.

3. **Requirement** `cr_7f62f6f22c9d`: Once WDOG_REGWEN.regwen is cleared, the watchdog configuration is locked until the next reset.
   Section: hw/ip/aon_timer/data/aon_timer.hjson > aon_timer > WDOG_REGWEN

   - `hw/ip/aon_timer/data/aon_timer.hjson:259`: `Once cleared, the watchdog configuration will be locked until the next reset.`

Tested by `WatchdogLockTest.Success`**:** The DIF lock function is checked to write 0 to WDOG_REGWEN, which is the action that clears regwen.
Not tested: No check that the watchdog configuration registers are actually locked afterwards, or that they stay locked until reset. This is a mocked DIF unit test, not hardware.

Tested by `WatchdogStartTest.Locked`**:** With REGWEN mocked to read 0, the DIF start call is checked to return kDifLocked, so software refuses to reconfigure a locked watchdog.
Not tested: The hardware write blocking of the watchdog registers is not verified. Only the software-side lock check against a mocked REGWEN is exercised.

Tested by `WatchdogStopTest.Locked`**:** With REGWEN mocked to read 0, the DIF stop call is checked to return kDifLocked.
Not tested: The hardware locking of the watchdog registers is not verified. Only the software-side lock check against a mocked REGWEN is exercised.

Tested by `WatchdogRestartTest.Locked`**:** With REGWEN mocked to read 0, the DIF restart call is checked to return kDifLocked.
Not tested: The hardware locking of the watchdog registers is not verified. Only the software-side lock check against a mocked REGWEN is exercised.

Tested by `WatchdogStartTest.SuccessLock`**:** watchdog_start with lock set is checked to write 0 to REGWEN after enabling the watchdog.
Not tested: No check that the configuration is locked after REGWEN is cleared, or that the lock persists until reset.

Tested by `WatchdogIsLockedTest.SuccessLocked`**:** A REGWEN read of 0 is checked to make watchdog_is_locked report true.
Not tested: No check that writes to the watchdog configuration are blocked in hardware, or that the lock lasts until reset.

Verified: DV does not write WDOG_REGWEN, and no hardware test checks the lock.

- `hw/ip/aon_timer/dv/env/seq_lib/aon_timer_base_vseq.sv:177`: `// csr_utils_pkg::csr_wr(ral.wdog_regwen, wdog_regwen);` — DV never clears WDOG_REGWEN, so locking is not exercised.

Items 2, 5 of this group were not verified (see Withdrawn and Inconclusive).

Searched:

- regwen use in DV and sw tests in `hw/ip/aon_timer/dv, sw/device/tests`: The regwen write is commented out in DV. No sw test uses WDOG_REGWEN.
- wake cause check in sleep tests in `sw/device/tests`: pwrmgr_sleep_all_wake_ups_impl.c checks the AON timer wakeup cause.
- bark threshold stimulus in `aon_timer dv seq_lib`: Randomized thresholds, plus min and max directed sequences.

Looked at and ruled out:

- `hw/ip/aon_timer/dv/env/seq_lib/aon_timer_base_vseq.sv:170`: Writes the bark threshold only; it does not test the regwen lock.

5.2 Partly tested: 1 requirement in programmers_guide.md > Initialization (and 3 more sections)

`f_760d5c3b21cf` · severity high

1. **Requirement** `cr_05e1d4dc8418`: Set WKUP_COUNT_LO, WKUP_COUNT_HI and WDOG_COUNT to zero as the first initialization step.
   Section: hw/ip/aon_timer/doc/programmers_guide.md > Programmer's Guide > Initialization

   - `hw/ip/aon_timer/doc/programmers_guide.md:5`: `Set the timer values WKUP_COUNT_LO, WKUP_COUNT_HI and WDOG_COUNT to zero.`

   Tested by `WatchdogStartTest.Success`**:** watchdog_start's register sequence zeroes the count (disable, zero count, write thresholds, then enable), which matches the zero-the-count step for the watchdog.
   Not tested: WKUP_COUNT_LO and WKUP_COUNT_HI are not covered. The zero-count write to WDOG_COUNT is only described in the check summary, not shown explicitly.

   Tested by `WakeupStartTest.Success`**:** wakeup_start disables the timer and writes 0 to the WKUP count registers before setting the prescaler and enabling, which matches the zero-the-count step for the wakeup timer.
   Not tested: WDOG_COUNT is not zeroed or checked, and the k2 check shown only cites WKUP_COUNT_LO, with no explicit check of WKUP_COUNT_HI.

   Verified: No test zeroes WKUP_COUNT_LO, WKUP_COUNT_HI and WDOG_COUNT as the first initialization step. The DV sequences write random counts.

   - `hw/ip/aon_timer/dv/env/seq_lib/aon_timer_base_vseq.sv:164`: `csr_utils_pkg::csr_wr(ral.wdog_count, wdog_count);` — WDOG_COUNT is written with a random value, not zero. This is why the first item is not covered.

Items 2, 3, 4, 5 of this group were not verified (see Withdrawn and Inconclusive).

Searched:

- DV coverage of the thresholds, WKUP_CAUSE clear and WDOG_COUNT writes in `hw/ip/aon_timer/dv/env`: The scoreboard and base vseq cover the hi/lo thresholds, the WKUP_CAUSE write of 0 and read check, and WDOG_COUNT writes.
- Zero-the-counts initialization step in `hw/ip/aon_timer/dv/env/seq_lib`: The sequences write random counts. No explicit zeroing of all three counts was found.

5.3 Partly tested: 2 requirements in theory_of_operation.md > Theory of Operation (and 1 more section)

`f_786c4113c34d` · severity high

1. **Requirement** `cr_20f39fd89cf5`: Both timers halt counting when lc_escalate_en_i is set.
   Section: hw/ip/aon_timer/doc/theory_of_operation.md > Theory of Operation

   - `hw/ip/aon_timer/doc/theory_of_operation.md:5`: `Both timers halt the counting if the signal `lc_escalate_en_i` is set`

Tested by `aon_timer_wdog_lc_escalate_test`**:** The test programs the watchdog bark and bite inside escalation phase 1 and forces escalation. It expects the reset reason to be escalation, which implies the watchdog halted.
Not tested: Only the WDOG timer is exercised. The bark interrupt not firing is only logged (k7, only reports) and does not fail the test. The WKUP timer halting is not checked.

Verified: No test asserts escalation while both timers run and checks that both counts freeze; the DV only drives it at reset and then clears it.

- `hw/ip/aon_timer/dv/env/seq_lib/aon_timer_base_vseq.sv:208-212`: `cfg.lc_escalate_en_vif.drive(0);` — Escalation is cleared right after init; it is driven only at reset with a random initial value, so there is no escalation asserted while both timers run, and no check on WKUP halting.

2. **Requirement** `cr_7aa0f0735984`: wkup_req_o and aon_timer_rst_req_o are driven on the AON domain. Section: hw/ip/aon_timer/doc/theory_of_operation.md > Theory of Operation

   - `hw/ip/aon_timer/doc/theory_of_operation.md:9`: `The output signals \`wkup_req_o\` and \`aon_timer_rst_req_o\` are driven on the AON domain.`

   Tested by `aon_timer_prescaler`**:** The test enables only the wakeup timer and waits for its interrupt. The scoreboard checks the wkup_req pin sampled through the AON interrupt interface when the wakeup timer expires.
   Not tested: aon_timer_rst_req_o is never exercised because the watchdog is disabled. Nothing checks that the outputs are driven in the AON clock domain.

   Tested by `aon_timer_jump`**:** The test enables both timers with counts near their thresholds. The scoreboard checkers then compare the wakeup request pin and the watchdog reset request pin, both sampled through the AON interrupt interface, against the model's predictions.
   Not tested: Nothing checks that the outputs are driven in the AON clock domain, such as timing against the AON clock. The checkers only compare pin values with predictions. It is also not clear that the bite path is reached every run.

   Verified: The scoreboard compares pin values with predictions, but nothing checks that the outputs are synchronous to the AON clock.

Item 3 of this group was not verified (see Withdrawn and Inconclusive).

Searched:

- lc_escalate in aon_timer dv in `hw/ip/aon_timer/dv`: Only driven at reset or cleared; no scoreboard model of halting.
- 64-bit thold/count writes in `hw/ip/aon_timer/dv/env`: Random 64-bit wkup_thold/count with HI/LO writes and scoreboard checks.

Looked at and ruled out:

- `hw/ip/aon_timer/dv/env/seq_lib/aon_timer_wkup_count_cdc_hi_vseq.sv:55-81`: Touches escalation and the HI count but does not check the halt.

5.4 Test plan items with no test: 3 plan items

`f_1e82ff493547` · severity medium

2. **Plan item:** mem_walk — Verify accessibility of all memories by running the standard UVM mem walk sequence on all memories in the RAL model, from all available interfaces. (`hw/dv/tools/dvsim/testplans/mem_testplan.hjson`)

   Verified: The generic mem_walk test is imported, but aon_timer has no memories, so it checks nothing.

   - `hw/dv/tools/dvsim/tests/mem_tests.hjson:22-25`: `name: "{name}_mem_walk"` — Generic mem_walk test is imported, but aon_timer has no memories, so the walk checks nothing

3. **Plan item:** SIM_FUNCTIONAL_COVERAGE_AT_100 — V3 checklist item: simulation functional coverage at 100%; not started. (`hw/ip/aon_timer/doc/checklist.md`)

   Verified: This is a checklist status entry (Not Started), not something a test can implement.

   - `hw/ip/aon_timer/doc/checklist.md:243`: `Coverage | [SIM_FUNCTIONAL_COVERAGE_AT_100][]| Not Started |` — Status entry only, not a test

4. **Plan item:** SEC_CM_DV_REVIEWED — Review checklist item for V2S, recorded as Done; the V2S review meeting was waived because only one standard sec_cm (bus integrity) exists. (`hw/ip/aon_timer/doc/checklist.md`)

   Verified: This is a review checklist item, not testable behavior.

Items 1, 4 of this group were not verified (see Withdrawn and Inconclusive).

Searched:

- X assertions in `hw/ip/aon_timer/rtl`: ASSERT_KNOWN on outputs
- tl_intg_err and mem_walk tests in `aon_timer_sim_cfg.hjson and the common tests/*.hjson files`: both are imported; aon_timer has no memories for mem_walk

---

5.5 Test plan items with no test: 1 plan item

`f_2f68d49be624` · severity medium

1. **Plan item:** X_PROP_ANALYSIS_COMPLETED — V3 checklist item: X-propagation analysis is completed; resolution Not Started. (`hw/ip/aon_timer/doc/checklist.md`)

   Verified: Resolution is Not Started and no X-prop test or analysis was found.

   - `hw/ip/aon_timer/doc/checklist.md:239`: `Tests | [X_PROP_ANALYSIS_COMPLETED][] | Not Started |` — X-prop analysis has not been started.

Items 2, 3, 4 of this group were not verified (see Withdrawn and Inconclusive).

Searched:

- X_PROP, pending_req_on_rst, SEC_CM_TESTPLAN, tlul_assert in aon_timer in `hw/ip/aon_timer`: checklist rows, README mentions, and the tlul_assert bind
- pending_req_on_rst in `hw/dv`: covergroup in tl_agent_cov.sv, sampled in tl_monitor.sv

5.6 Test plan items with no test: 4 plan items

`f_5c298f797c68` · severity medium

1. **Plan item:** SIM_CODE_COVERAGE_AT_100 — V3 checklist item: simulation code coverage at 100%; not started. (`hw/ip/aon_timer/doc/checklist.md`)

   Verified: The checklist item is Not Started and is a process item with no test.

   - `hw/ip/aon_timer/doc/checklist.md:241-242`: `Regression | [SIM_NIGHTLY_REGRESSION_AT_V3] [] | Not Started |` — V3 items are Not Started, so no test exists for them.

2. **Plan item:** SIM_NIGHTLY_REGRESSION_AT_V3 — V3 checklist item: nightly simulation regression at V3; resolution Not Started. (`hw/ip/aon_timer/doc/checklist.md`)

   Verified: The checklist item is Not Started and is a process item with no test.

3. **Plan item:** SIM_COVERAGE_REVIEWED — V2S coverage checklist item for reviewing simulation coverage, resolution Done. (`hw/ip/aon_timer/doc/checklist.md`)

   Verified: This is a coverage review activity, not a test.

4. **Plan item:** tl_a_chan_cov_cg — Reused TL A channel coverage of mask, opcode, size, source and their cross. (`hw/ip/aon_timer/dv/README.md`)

   Verified: This is a reused covergroup from the common TL agent with no dedicated test. Any test that drives TL traffic samples it.

   - `hw/ip/aon_timer/dv/README.md:142`: `Covergroup \`tl_a_chan_cov_cg\`:` — A reused covergroup from the common TL agent, not a test.

Item 4 of this group was not verified (see Withdrawn and Inconclusive).

Searched:

- SIM_CODE_COVERAGE_AT_100 status and any test in `hw/ip/aon_timer/doc/checklist.md`: Not Started; process item, no test
- SIM_NIGHTLY_REGRESSION_AT_V3 status and any test in `hw/ip/aon_timer/doc/checklist.md`: Not Started; process item, no test
- SIM_COVERAGE_REVIEWED in `hw/ip/aon_timer/doc/checklist.md`: Done; a review activity, no test
- SIM_SEC_CM_VERIFIED and sec_cm tests in `hw/ip/aon_timer/data/aon_timer_sec_cm_testplan.hjson and dv/aon_timer_sim_cfg.hjson`: sec_cm_bus_integrity maps to the test aon_timer_tl_intg_err; sec_cm_tests.hjson is imported in the sim cfg
- tl_a_chan_cov_cg in `hw/ip/aon_timer/dv`: Only mentioned in the README as a reused covergroup; no dedicated test

Looked at and ruled out:

- `hw/ip/aon_timer/data/aon_timer_sec_cm_testplan.hjson:27-31`: This is the test that backs SIM_SEC_CM_VERIFIED, so that item is not unimplemented.

5.7 Test plan items with no test: 2 plan items

`f_621472fe1b41` · severity medium

1. **Plan item:** mem_partial_access — Verifies partial accessibility of all memories via partial reads and writes, and tests outstanding accesses on memories. (`hw/dv/tools/dvsim/testplans/mem_testplan.hjson`)

   Verified: The shared test exists but aon_timer has no memory, so no stimulus or check applies.

   - `hw/dv/tools/dvsim/tests/mem_tests.hjson:29`: `name: "{name}_mem_partial_access"` — Test exists but aon_timer has no memory, so it checks nothing.

2. **Plan item:** FPV_CODE_COVERAGE_AT_100 — V3 checklist item: FPV code coverage at 100%; not started. (`hw/ip/aon_timer/doc/checklist.md`)

   Verified: The checklist marks it Not Started and no FPV code coverage test exists.

   - `hw/ip/aon_timer/doc/checklist.md:244`: `Coverage | [FPV_CODE_COVERAGE_AT_100][] | Not Started |` — The FPV coverage item is explicitly not started.

Items 3, 4, 5 of this group were not verified (see Withdrawn and Inconclusive).

Searched:

- csr_aliasing and rand_reset tests in `hw/dv/tools/dvsim/tests/csr_tests.hjson, aon_timer sim cfg`: Both are defined as shared tests and imported by aon_timer.
- mem tests and memories in aon_timer in `mem_tests.hjson, aon_timer.hjson, aon_timer_common_vseq.sv`: The shared mem_partial_access test is imported, but the design has no memory, so it is a no-op.
- dv_base_lockable_field_cov in `hw/dv`: The covergroup is in dv_base_reg and the scoreboard tracks wdog_regwen, but I did not confirm that it is sampled for aon_timer.

Looked at and ruled out:

- `hw/dv/sv/cip_lib/seq_lib/cip_base_vseq.sv:1435-1438`: The partial-access sequence loops over RAL memories, and aon_timer has none, so it verifies nothing.

5.8 Test plan items with no test: 1 plan item

`f_dc2b6322be7a` · severity medium

1. **Plan item:** FPV_ASSERTIONS_PROVEN_AT_V3 — V3 checklist item: FPV assertions are proven; resolution Not Started. (`hw/ip/aon_timer/doc/checklist.md`)

   Verified: The checklist item is Not Started and I found no FPV tests or assertions for it.

   - `hw/ip/aon_timer/doc/checklist.md:240`: `[FPV_ASSERTIONS_PROVEN_AT_V3][] | Not Started |` — The checklist item is marked Not Started.

Items 2, 3, 4, 5 of this group were not verified (see Withdrawn and Inconclusive).

Searched:

- csr_bit_bash and tl_intg_err in the dvsim test lists and in the aon_timer sim cfg in `hw/dv/tools/dvsim, hw/ip/aon_timer/dv`: aon_timer_sim_cfg.hjson imports csr_tests.hjson and tl_access_tests.hjson, which define both tests.
- intr_pins_cg and max_outstanding_cg in `hw/`: intr_pins_cg is sampled in the aon_timer scoreboard. I did not find where either covergroup is defined, and I did not check max_outstanding_cg further.

5.9 Test plan items with no test: 1 plan item

`f_e8981726353a` · severity medium

1. **Plan item:** FPV_COI_COVERAGE_AT_100 — V3 checklist item: FPV cone-of-influence coverage at 100%; not started. (`hw/ip/aon_timer/doc/checklist.md`)

   Verified: The checklist marks it Not Started and no formal collateral was found.

   - `hw/ip/aon_timer/doc/checklist.md:245`: `Coverage | [FPV_COI_COVERAGE_AT_100][] | Not Started |` — This is a checklist item that is not started; no test exists.

Items 2, 3, 4, 5 of this group were not verified (see Withdrawn and Inconclusive).

Searched:

- alert_test in the aon_timer sim cfg and the common test list in `hw/ip/aon_timer/dv, hw/dv/tools/dvsim/tests`: alert_test.hjson is imported and defines the test {name}_alert_test.
- covergroup definitions and README mentions in `hw/dv/sv/cip_lib, hw/ip/aon_timer/dv/README.md`: tl_errors_cg_wrap and tl_intg_err_cg_wrap exist in cip_base_env_cov.sv and are documented in the README.
- FPV items in the checklist in `hw/ip/aon_timer/doc/checklist.md`: FPV_SEC_CM_VERIFIED is N/A. FPV_COI_COVERAGE_AT_100 is Not Started.

5.10 Partly tested: 1 requirement in aon_timer.hjson > WDOG_BITE_THOLD

`f_57c50a9573d0` · severity medium

1. **Requirement** `cr_dce064574acf`: A watchdog bite triggers a reset; the bite reset is generated when the watchdog count reaches WDOG_BITE_THOLD.

   - `hw/ip/aon_timer/data/aon_timer.hjson:303`: `The count at which a watchdog bite reset should be generated.`
   - `hw/ip/aon_timer/data/aon_timer.hjson:153`: `A bite triggers a reset.`

   Tested by `aon_timer_jump`**:** Writes the watchdog count to within 10 of the bark/bite thresholds and enables the watchdog, so the count reaches the bite threshold; scoreboard c8 checks the reset_req pin against the modelled expectation.
   Not tested: The test has no checks of its own and only exercises counts near the threshold; whether the bite is actually hit depends on the randomisation, and only the scoreboard's model verifies the reset.

   Tested by `aon_timer_smoke_min_thold`**:** Runs the smoke sequence with the bite threshold constrained to 0, with the scoreboard model_and_check_wdog_bite (c8) checking the reset request pin against the modelled bite expectation.

Not tested: Only the minimum threshold value (0) is exercised; mid-range thresholds are not covered here, and the reset request is only checked through the scoreboard model.

Tested by `aon_timer_smoke_max_thold`**:** Runs the smoke sequence with the bite threshold at its maximum value, with scoreboard c8 checking the reset request pin against the modelled bite.
Not tested: Only the maximum threshold value is exercised, and the bite may not be reached at that value in simulation.

Tested by `aon_timer_wdog_lc_escalate_test`**:** Programs the watchdog bark and bite timeouts and expects an escalation reset; the reset reason check is for escalation, not a watchdog bite.
Not tested: Does not check that the bite reset is generated when the count reaches the threshold; the test checks escalation resets instead.

Verified: The scoreboard checks the pin only at the modelled expiry. The deterministic mid-range threshold and the check that the pin is not asserted early are both missing.

- `hw/ip/aon_timer/dv/env/aon_timer_scoreboard.sv:1380-1393`: `wdog_rst_req_exp = 1'b1;` — The reset_req pin is compared with the expected value once, after the modelled count reaches wdog_bite_num. No check of the pin before that point turned up in this function.

Searched:

- bite handling in the aon_timer DV directory in `hw/ip/aon_timer/dv`: The scoreboard model_and_check_wdog_bite is the only reset_req check that turned up. Sequences for jump, min_thold and max_thold only configure stimulus.

Looked at and ruled out:

- `hw/ip/aon_timer/dv/env/seq_lib/aon_timer_jump_vseq.sv:1`: I did not read this file. Per the finding it only puts the count near the thresholds, so it does not deterministically hit a mid-range bite.

5.11 No test found: 4 requirements in aon_timer.hjson > inter_signal_list (and 2 more sections)

`f_96eff24db6b6` · severity medium

1. **Requirement** `cr_068211cfcae2`: racl_error outputs the RACL error log information of this module.
   Section: hw/ip/aon_timer/data/aon_timer.hjson > aon_timer > inter_signal_list

   - `hw/ip/aon_timer/data/aon_timer.hjson:126`: `RACL error log information of this module.`

   Verified: racl_error_o is left unconnected in tb.sv, so nothing checks it.

   - `hw/ip/aon_timer/dv/tb.sv:62`: `.racl_error_o (),` — racl_error output is unconnected, so it is never checked.

2. **Requirement** `cr_1525c2bff7eb`: racl_policies receives the incoming RACL policy vector from a racl_ctrl instance.
   Section: hw/ip/aon_timer/data/aon_timer.hjson > aon_timer > inter_signal_list

   - `hw/ip/aon_timer/data/aon_timer.hjson:115`: `Incoming RACL policy vector from a racl_ctrl instance.`

Verified: The policies are tied to 0, so there is no stimulus.

- `hw/ip/aon_timer/dv/tb.sv:25-26` — RACL policies tied to zero, so no policy stimulus.

```
racl_policy_vec_t racl_policies;
  assign racl_policies = 0;
```

3. **Requirement** `cr_d3bb7e9459e0`: The policy selection vector parameter selects the RACL policy applied to each register.
   Section: hw/ip/aon_timer/data/aon_timer.hjson > aon_timer > inter_signal_list

   - `hw/ip/aon_timer/data/aon_timer.hjson:116`: `The policy selection vector (parameter) selects the policy for each register.`
   - `hw/ip/aon_timer/doc/interfaces.md:20`: `The policy selection vector (parameter) selects the policy for each register.`

   Verified: No RACL policy variation or checking exists in the DV environment.

4. **Requirement** `cr_7352e4c71220`: WKUP_CTRL.prescaler: hardware receives a write-qualifier pulse on software writes (true).
   Section: hw/ip/aon_timer/data/aon_timer.hjson > aon_timer > WKUP_CTRL

   - `hw/ip/aon_timer/data/aon_timer.hjson:196`: `hwqe: "true"`

   Verified: The scoreboard has no check on the prescaler write-qualifier pulse. It only models the prescaler value.

   - `hw/ip/aon_timer/dv/env/aon_timer_scoreboard.sv:471-474`: `task aon_timer_scoreboard::update_wkup_thold_hi_timely();` — The scoreboard models THOLD_HI writes (via the _we probe) but nothing checks the bit-[63:32] split explicitly.

Item 5 of this group was not verified (see Withdrawn and Inconclusive).

Searched:

- racl in aon_timer dv in `hw/ip/aon_timer/dv`: Only tb tie-off and a coverage exclusion file; no tests or checks.
- prescaler and thold_hi in the scoreboard in `aon_timer_scoreboard.sv`: The scoreboard models both registers, but it does not check the prescaler hwqe pulse. It does probe the thold_hi _we signal.

5.12 No test found: 5 requirements in interfaces.md > Interfaces (and 1 more section)

`f_1e00f67be20c` · severity medium

1. **Requirement** `cr_627c824e7226`: The aon_timer module has no TL-UL bus host interfaces.
   Section: hw/ip/aon_timer/doc/interfaces.md > Interfaces

   - `hw/ip/aon_timer/doc/interfaces.md:8`: `Bus Host Interfaces (TL-UL): *none*`

   Verified: A structural declaration; no test checks it.

   - `hw/ip/aon_timer/dv/tb.sv:25-26` — RACL policies tied off to 0, so no stimulus reaches the RACL path

```
racl_policy_vec_t racl_policies;
  assign racl_policies = 0;
```

2. **Requirement** `cr_8d65e39a5db1`: The aon_timer module has no peripheral pins for chip IO.
   Section: hw/ip/aon_timer/doc/interfaces.md > Interfaces

   - `hw/ip/aon_timer/doc/interfaces.md:9`: `Peripheral Pins for Chip IO: *none*`

   Verified: A structural declaration; no test checks it.

3. **Requirement** `cr_e4e318bad395`: The aon_timer module has primary clock clk_i.
   Section: hw/ip/aon_timer/doc/interfaces.md > Interfaces

   - `hw/ip/aon_timer/doc/interfaces.md:5`: `Primary Clock: **`clk_i`**`

   Verified: The clock is used by the testbench, but no test checks the declaration itself.

4. **Requirement** `cr_5867b0a56d13`: racl_policies is a 1-bit top_racl_pkg::racl_policy_vec unidirectional input (receiver).
   Section: hw/ip/aon_timer/doc/interfaces.md > Interfaces > Inter-Module Signals

   - `hw/ip/aon_timer/doc/interfaces.md:20`: `racl_policies top_racl_pkg::racl_policy_vec uni rcv 1`

   Verified: racl_policies is tied to 0 in the testbench, so it is never exercised.

   - `hw/ip/aon_timer/dv/cov/aon_timer_unr_non_racl_manually_excluded_excl.el:14`: `ANNOTATION: "This is a non-racl instance - hence signals won't toggle."` — RACL signals are excluded from coverage because they do not toggle

5. **Requirement** `cr_c6cf966d6f9f`: racl_error is a 1-bit top_racl_pkg::racl_error_log unidirectional output (requester).
   Section: hw/ip/aon_timer/doc/interfaces.md > Interfaces > Inter-Module Signals

   - `hw/ip/aon_timer/doc/interfaces.md:21`: `racl_error top_racl_pkg::racl_error_log uni req 1`

   Verified: racl_error is left unconnected and never checked.

   - `hw/ip/aon_timer/dv/tb.sv:62`: `.racl_error_o (),` — racl_error output is left unconnected and never checked

Searched:

- racl in aon_timer dv (racl_policies and racl_error, item 4 and 5) in `hw/ip/aon_timer/dv`: Only the tb tie-off, an unconnected racl_error_o and coverage exclusions; no test or checker
- TL-UL host interface and chip IO pins (item 1 and 2) in `hw/ip/aon_timer/dv and the hjson-derived doc`: Structural 'none' declarations; no test or assertion checks them
- primary clock clk_i (item 3) in `hw/ip/aon_timer/dv/tb.sv`: The clock is driven in the testbench, but no check verifies the declaration itself

---

5.13 No test found: 5 requirements in programmers_guide.md > Writing the counter (and 2 more sections)

`f_1a5ce3143a3c` · severity medium

1. **Requirement** `cr_ebc0e38e9d47`: For target values close to overflow, disable the timer via WKUP_CTRL while writing the counter.
   Section: hw/ip/aon_timer/doc/programmers_guide.md > Programmer's Guide > Wakeup count and threshold access > Writing the counter

   - `hw/ip/aon_timer/doc/programmers_guide.md:56`: `For such values, disable the timer with WKUP_CTRL while writing the counter.`

   Verified: No test disables the timer through WKUP_CTRL while writing a near-overflow counter value.

   - `hw/ip/aon_timer/dv/env/seq_lib/aon_timer_base_vseq.sv:194`: `write_wkup_reg(ral.wkup_thold_lo, wkup_thold[31:0]);` — A single THOLD_LO write with the final value. There is no UINT32_MAX-first write and no interrupt disable and re-enable around it.

2. **Requirement** `cr_13a3d2eb44cf`: Hardware does not modify WKUP_THOLD_LO and WKUP_THOLD_HI.
   Section: hw/ip/aon_timer/doc/programmers_guide.md > Programmer's Guide > Wakeup count and threshold access > Reading the threshold

   - `hw/ip/aon_timer/doc/programmers_guide.md:60`: `The hardware does not alter [`WKUP_THOLD_LO`](registers.md#wkup_thold_lo) and [`WKUP_THOLD_HI`](registers.md#wkup_thold_hi)`

   Verified: No check that the hardware never modifies THOLD_LO or THOLD_HI was found.

3. **Requirement** `cr_498c4b121026`: Software first writes WKUP_THOLD_LO with its maximum value (UINT32_MAX).
   Section: hw/ip/aon_timer/doc/programmers_guide.md > Programmer's Guide > Wakeup count and threshold access > Writing the threshold

   - `hw/ip/aon_timer/doc/programmers_guide.md:70`: `// Set the lower word to its maximum value.`
   - `hw/ip/aon_timer/doc/programmers_guide.md:72`: `REG_WRITE(WKUP_THOLD_LO, UINT32_MAX);`

   Verified: No test writes THOLD_LO with UINT32_MAX first.

4. **Requirement** `cr_64e15132845f`: Software then writes WKUP_THOLD_LO with the lower 32 bits of the new threshold.
   Section: hw/ip/aon_timer/doc/programmers_guide.md > Programmer's Guide > Wakeup count and threshold access > Writing the threshold

   - `hw/ip/aon_timer/doc/programmers_guide.md:78-79`: `// Set the lower word to its desired value. REG_WRITE(WKUP_THOLD_LO, new_thold & 0xffff_ffff);`

   Verified: The single THOLD_LO write exists, but it is not the second step of the guide's sequence and nothing checks the sequence.

5. **Requirement** `cr_ab0d74b3889d`: Software re-enables the wakeup interrupt after the threshold update is complete.
   Section: hw/ip/aon_timer/doc/programmers_guide.md > Programmer's Guide > Wakeup count and threshold access > Writing the threshold

   - `hw/ip/aon_timer/doc/programmers_guide.md:81`: `enable_wakeup_interrupt();`

   Verified: No test re-enables the wakeup interrupt after a threshold update.

Searched:

- item 1: counter writes near overflow with the timer disabled via WKUP_CTRL in `hw/ip/aon_timer/dv (wkup_count, 0xffff_ffff)`: Only the scoreboard's timing tracking of count writes and a backdoor read after a disable and re-enable. No near-overflow disable procedure.
- item 2: check that the hardware never changes THOLD_LO or THOLD_HI in `hw/ip/aon_timer/dv, sw/device`: No such check. The scoreboard only tracks the timing of the write enables.
- item 3: UINT32_MAX written to THOLD_LO first in `sw/device (aon_timer tests and DIF), hw/ip/aon_timer/dv`: UINT32_MAX appears only in watchdog config calls. The DIF writes THOLD_LO once.
- item 4: THOLD_LO written with the low 32 bits as the second step in `hw/ip/aon_timer/dv, DIF`: Only a single write. The two-step sequence is neither driven nor checked.
- item 5: wakeup interrupt re-enabled after the threshold update in `hw/ip/aon_timer/dv/env/seq_lib (intr_enable, intr_state)`: The sequences only read or clear intr_state or use intr_test. None re-enable the interrupt after a threshold update.

Looked at and ruled out:

- `hw/ip/aon_timer/dv/env/aon_timer_scoreboard.sv:462-469`: It only tracks the timing of the THOLD_LO write. It does not check the guide's write sequence or that the hardware leaves the register unchanged.

5.14 No test found: 2 requirements in programmers_guide.md > Writing the threshold (and 1 more section)

`f_900bad7fdee4` · severity medium

2. **Requirement** `cr_cd988f27543a`: Software disables the wakeup interrupt before writing the threshold registers.
   Section: hw/ip/aon_timer/doc/programmers_guide.md > Programmer's Guide > Wakeup count and threshold access > Writing the threshold

   - `hw/ip/aon_timer/doc/programmers_guide.md:68`: `disable_wakeup_interrupt();`

   Verified: No sequence disables the wakeup interrupt before writing the threshold registers.

   - `hw/ip/aon_timer/dv/env/seq_lib/aon_timer_base_vseq.sv:194-196` — Item 1: the upper word is written in wkup_init, which every test runs. It is written after the lower word with no interrupt disable beforehand (item 2).

```
write_wkup_reg(ral.wkup_thold_lo, wkup_thold[31:0]);
if (cfg.under_reset) return;
write_wkup_reg(ral.wkup_thold_hi, wkup_thold[63:32]);
```

3. **Requirement** `cr_bb10680d5ea9`: If the count remains above the threshold after clearing and the timer stays enabled, the wakeup event triggers again at the next clock tick.
   Section: hw/ip/aon_timer/doc/programmers_guide.md > Programmer's Guide > Interrupt Handling

   - `hw/ip/aon_timer/doc/programmers_guide.md:93`

     ```
     If {[`WKUP_COUNT_HI`](registers.md#wkup_count_hi), [`WKUP_COUNT_LO`]
     (registers.md#wkup_count_lo)} remains above the threshold after clearing the
     interrupt or wakeup event and the timer remains enabled, the interrupt and wakeup
     event will trigger again at the next clock tick.
     ```

   Verified: The scoreboard has no check that the interrupt re-triggers after a clear while the count remains above the threshold.

   - `hw/ip/aon_timer/dv/env/aon_timer_scoreboard.sv:706-709` — On an intr_state clear the scoreboard just drops the expected interrupt. No explicit re-trigger-after-clear check for count above the threshold.

     ```
     if (intr_state_val[WKUP]) begin
       intr_status_exp[WKUP] = 1'b0;
       update_prediction = 1;
     ```

Item 1 of this group was not verified (see Withdrawn and Inconclusive).

Searched:

- THOLD_HI writes in DV in `hw/ip/aon_timer/dv`: wkup_init writes thold_hi; scoreboard predicts and checks
- intr_enable usage in sequences in `aon_timer/dv/env/seq_lib`: none, except custom_intr coverage sequence
- retrigger/again after clear in `aon_timer excluding rtl`: only the doc line; no test

Looked at and ruled out:

- `hw/ip/aon_timer/dv/env/seq_lib/aon_timer_jump_vseq.sv:18-28`: Starts the count near the threshold but does not clear the interrupt and check that it fires again.

5.15 No test found: 2 requirements in theory_of_operation.md > Design Details

`f_1de3c49add8e` · severity medium

1. **Requirement** `cr_9958ce5b3109`: Reading back a register guarantees completion of all previous writes to the peripheral.

   - `hw/ip/aon_timer/doc/theory_of_operation.md:27-28`: `If software needs to guarantee completion of a register write then it can read back the register value. This will guarantee the completion of all previous writes to the peripheral.`

Verified: No test uses readback to confirm that earlier writes have completed.

2. **Requirement** `cr_bfbda21a02ad`: When integrating the AON timer, clk_aon_i must be started before clk_i.

   - `hw/ip/aon_timer/doc/theory_of_operation.md:36`: `When integrating the AON timer into a bigger design, you must start the always-on clock (clk_aon_i) before you start the main clock (clk_i).`

   Verified: Clock start order is an integration requirement and no DV test checks it.

Item 3 of this group was not verified (see Withdrawn and Inconclusive).

Searched:

- readback / clk_aon ordering in aon_timer dv in `hw/ip/aon_timer`: Only the doc lines. No test or assertion covers them.
- wkup_thold_lo in dv in `hw/ip/aon_timer/dv`: Scoreboard modelling, a base vseq write, and generic CSR tests.

Looked at and ruled out:

- `hw/ip/aon_timer/dv/env/aon_timer_scoreboard.sv:462-469`: Timing of the thold_lo write is modelled, but it does not explicitly verify bits 31:0 of the threshold.

5.16 Only weak evidence (no check that can fail): 2 requirements in theory_of_operation.md > Design Details

`f_53c338cbaca4` · severity medium

1. **Requirement** `cr_874f04d5bc14`: The watchdog timer is 32 bits wide.

   - `hw/ip/aon_timer/doc/theory_of_operation.md:21`: `the watchdog timer is 32 bits wide`

   Nearest evidence (not counted as a test): `aon_timer_smoke_max_thold` (stimulus only, no check)

   Verified: No explicit 32-bit watchdog width check was found. The max-threshold test only sets the thresholds to their maximum values, and the scoreboard predicts interrupts from a 32-bit count.

   - `hw/ip/aon_timer/dv/env/aon_timer_scoreboard.sv:933`: `if (wkup_count <= wkup_thold) begin` — The scoreboard uses the 64-bit count only to predict when the interrupt fires. This is an indirect check at most, with no explicit width or carry assertion.

2. **Requirement** `cr_ae28e3f9f608`: The wakeup timer is 64 bits wide.

   - `hw/ip/aon_timer/doc/theory_of_operation.md:21`: `The wakeup timer is 64 bits wide`

   Nearest evidence (not counted as a test): `aon_timer_wkup_count_cdc_hi` (stimulus only, no check)

   Verified: The cdc_hi test drives the low-half overflow, but no check was found that the carry reaches the high half or that the counter is 64 bits wide. The scoreboard's interrupt prediction is only an indirect check.

- `hw/ip/aon_timer/dv/env/seq_lib/aon_timer_wkup_count_cdc_hi_vseq.sv:5-6`: `// aon_timer_wkup_count_cdc_hi_vseq aims to get the wkup_count[31:0] to overflow` — The stimulus crosses the low-half overflow, but the stated purpose is CDC coverage closure, not a check of the carry or the width.

Searched:

- explicit width or carry checks in the scoreboard, and the cdc_hi vseq in `hw/ip/aon_timer/dv/env`: No explicit width or carry check. Only the 64-bit wkup_count and 32-bit wdog_count mirror variables used for interrupt-time prediction.

Looked at and ruled out:

- `hw/ip/aon_timer/dv/env/aon_timer_scoreboard.sv:774-782`: It mirrors the lo and hi halves into a 64-bit variable, but it does not assert that the counter carries from lo into hi or that the width is 64.

5.17 No test found: 2 requirements in theory_of_operation.md > Theory of Operation (and 1 more section)

`f_59063c90be4c` · severity medium

1. **Requirement** `cr_517c566609ad`: The RACL ports are enabled by setting parameter EnableRacl to 1. Section: hw/ip/aon_timer/doc/theory_of_operation.md > Theory of Operation

   - `hw/ip/aon_timer/doc/theory_of_operation.md:12`: `In order to enable the RACL ports, the parameter `EnableRacl` should be set to 1.`

   Verified: RACL is not enabled in the DV testbench, and the coverage for it is excluded.

   - `hw/ip/aon_timer/dv/tb.sv:25-26` — RACL policies are tied to 0, so no RACL stimulus is applied.

     ```
     racl_policy_vec_t racl_policies;
       assign racl_policies = 0;
     ```

   - `hw/ip/aon_timer/dv/cov/aon_timer_unr_non_racl_manually_excluded_excl.el:14`: `ANNOTATION: "This is a non-racl instance - hence signals won't toggle."` — The DV flow treats the instance as non-RACL and excludes the RACL coverage.

2. **Requirement** `cr_c98fa20138a9`: The block has RACL ports racl_policies_i and racl_error_o. Section: hw/ip/aon_timer/doc/theory_of_operation.md > Theory of Operation

   - `hw/ip/aon_timer/doc/theory_of_operation.md:11`: `Furthermore, the block also has RACL ports `racl_policies_i` and `racl_error_o``

   Verified: The RACL ports are only tied off; no test checks them.

Items 2, 3, 5 of this group were not verified (see Withdrawn and Inconclusive).

Searched:

- racl / EnableRacl / nmi in aon_timer dv in `hw/ip/aon_timer/dv`: RACL is tied off and its coverage excluded; nmi has a testbench assertion
- 200kHz / aon clock frequency in `hw/ip/aon_timer/dv`: no frequency check found

Looked at and ruled out:

- `hw/ip/aon_timer/dv/tb.sv:61-62`: The RACL ports are connected but tied off, so nothing exercises them.

Inconclusive

The verifier could neither confirm nor refute these. They need a human decision.

- `f_760d5c3b21cf` (partly tested) Writing zero to the WDOG_COUNT register pets the watchdog. — DV writes WDOG_COUNT and the scoreboard tracks the count update. I did not confirm a directed test that writes 0 to pet a running watchdog and then checks that bark and bite are delayed.
- `f_2f68d49be624` (test plan items with no test) SEC_CM_TESTPLAN_COMPLETED — This is a documentation checklist item marked Done, so it is not testable and has no implementing test to find.
- `f_621472fe1b41` (test plan items with no test) dv_base_lockable_field_cov — This is a coverage item, not a test. The covergroup exists in dv_base_reg and the regwen path is exercised, but I did not confirm it is sampled in aon_timer.
- `f_dc2b6322be7a` (test plan items with no test) intr_pins_cg — This is a coverage item, not a test. It is sampled in the scoreboard, so it is exercised by every test that runs it.
- `f_dc2b6322be7a` (test plan items with no test) max_outstanding_cg — This is a coverage item, not a test. I did not find where it is defined or sampled.
- `f_e8981726353a` (test plan items with no test) tl_errors_cg_wrap — The covergroup exists in the shared cip_lib, but I did not check where it is sampled or which tests drive it.
- `f_e8981726353a` (test plan items with no test) tl_intg_err_cg — The covergroup exists in the shared cip_lib, but I did not check where it is sampled or which tests drive it.
- `f_96eff24db6b6` (no test found) WKUP_THOLD_HI holds bits [63:32] of the 64-bit wakeup threshold. — The scoreboard models THOLD_HI writes and the timer interrupt timing, which likely exercises the high bits. I did not confirm an explicit check on the split.
- `f_1de3c49add8e` (no test found) WKUP_THOLD_LO.threshold_lo occupies bits 31:0 and holds wakeup threshold bits [31:0]. — The scoreboard and CSR tests exercise thold_lo indirectly. I did not confirm an explicit check of the bits 31:0 mapping.
- `f_59063c90be4c` (no test found) intr_wkup_timer_expired_o, intr_wdog_timer_bark_o and nmi_wdog_timer_bark_o are driven on the SYS domain. — I did not search thoroughly for a check that these outputs are driven on the SYS domain.
- `f_59063c90be4c` (no test found) The always-on timer runs on a ~200KHz clock. — I found no check on the ~200KHz clock frequency, but my search was limited.

Withdrawn

Candidates the verifier refuted, with the reason. Listed so the refutations can be checked too.

- `f_c7ed49ba3834` (spec too ambiguous to test) The cause bit indicates that the AON timer requested a wakeup. — The README says which events set the cause bit: the wakeup threshold match and the watchdog bark. (`hw/ip/aon_timer/README.md:35-44`, `hw/ip/aon_timer/README.md:69-77`)
- `f_c17ea0faef77` (spec too ambiguous to test) The prescaler field holds the pre-scaler value for the wakeup timer count. — The aon_timer README states that the number of cycles per tick is one more than the prescaler field. (`hw/ip/aon_timer/README.md:31-32`)

- `f_4f700106f7fc` (spec too ambiguous to test) A watchdog bark interrupt is generated when the watchdog count reaches the threshold value. — The aon_timer README states the bark condition: bark fires on every AON clock tick while the timer is enabled and the count is at least WDOG_BARK_THOLD. (`hw/ip/aon_timer/README.md:55`)
- `f_f2eacf1f7e83` (spec too ambiguous to test) Once WDOG_REGWEN.regwen is cleared, the watchdog configuration is locked until the next reset. — The register definitions say which registers WDOG_REGWEN locks. (`hw/ip/aon_timer/data/aon_timer.hjson:265-269`, `hw/ip/aon_timer/doc/registers.md:146`)
- `f_9254192820a9` (spec too ambiguous to test) A wakeup interrupt is generated when the count reaches the threshold value. — The README and programmer's guide say the condition is count at or above the threshold, checked on each prescaler tick. (`hw/ip/aon_timer/README.md:35`, `hw/ip/aon_timer/doc/programmers_guide.md:93`)
- `f_1bc5edd4d321` (spec too ambiguous to test) The count at which a wakeup interrupt should be generated, bottom 32 bits. — The README and programmer's guide say the wakeup interrupt fires when the count is at or above the threshold, on each prescaler tick, and re-triggers after a clear while the count stays above it. (`hw/ip/aon_timer/README.md:35`, `hw/ip/aon_timer/doc/programmers_guide.md:93`)
- `f_d5ddad4f6de4` (spec too ambiguous to test) The RACL ports are enabled by setting parameter EnableRacl to 1. — The general RACL document covers most of what the finding says is missing. (`doc/contributing/hw/racl/README.md:333-335`, `doc/contributing/hw/racl/README.md:29-30`)
- `f_6bf289f534a0` (spec contradicts itself) TL-UL register accesses from the SYS domain are synchronized to the AON slow clock using the async register generation feature. — The cited spec line says "Register reads ... (`hw/ip/aon_timer/doc/theory_of_operation.md:25-28`)
- `f_34248cf8b57c` (test contradicts its own description) aon_timer_prescaler: plan vs. test — The vseq never reads WKUP_COUNT, as the finding says. (`hw/ip/aon_timer/dv/env/aon_timer_scoreboard.sv:933-934`)
- `f_a5e981f8ba29` (test contradicts its own description) aon_timer_smoketest: plan vs. test — The plan item lists both pwrmgr_smoketest and aon_timer_smoketest as its bazel tests, with chip_sw_pwrmgr_smoketest as the DV test. (`hw/top_earlgrey/data/ip/chip_aon_timer_testplan.hjson:43-44`, `sw/device/tests/pwrmgr_smoketest.c:62`)
- `f_7dfb4807049d` (test contradicts its own description) check_intr_value_propagated: comment vs. code — wait_n_clks waits on the clocking block cbn, which is defined as @(negedge clk). (`hw/dv/sv/common_ifs/clk_rst_if.sv:116-118`, `hw/dv/sv/common_ifs/clk_rst_if.sv:107`)
- `f_978f56ce38cc` (test contradicts its own description) WatchdogStartTest.Success: comment vs. code — The finding read only a fragment of the test. (`sw/device/lib/dif/dif_aon_timer_unittest.cc:168-172`)
- `f_e4407a7a1bc0` (spec not tested) Partially tested: 3 requirement(s) in hw/ip/aon_timer/doc/programmers_guide.md > Programmer's Guide > Interrupt Handling — The finding only looked at mocked DIF/software unit tests. (`hw/ip/aon_timer/dv/env/aon_timer_scoreboard.sv:749-757`, `hw/ip/aon_timer/dv/env/aon_timer_scoreboard.sv:774-783`)
- `f_bf4e0a585a10` (spec not tested) 5 plan item(s) with no implementing test — All five items are implemented by shared infrastructure that aon_timer imports. (`hw/dv/tools/dvsim/tests/csr_tests.hjson:19-21`, `hw/dv/tools/dvsim/testplans/csr_testplan.hjson:102`)

- `f_dd95b1b85c7c` (spec not tested) Untested: 4 requirement(s) in hw/ip/aon_timer/data/aon_timer.hjson > aon_timer > INTR_TEST — The aon_timer sim config includes the generic intr_test test. (`hw/ip/aon_timer/dv/aon_timer_sim_cfg.hjson:33`, `hw/dv/sv/cip_lib/seq_lib/cip_base_vseq.sv:782-792`)
- `f_2354af605b15` (spec not tested) Planned but not implemented: 1 requirement(s) in hw/ip/aon_timer/data/aon_timer.hjson > aon_timer > features — The test is implemented and listed in the chip regressions, so it is not planned-only. (`sw/device/tests/aon_timer_wdog_lc_escalate_test.c:230-232`, `sw/device/tests/aon_timer_wdog_lc_escalate_test.c:276-278`)
- `f_b8451e043253` (spec not tested) Untested: 1 requirement(s) in hw/ip/aon_timer/doc/interfaces.md > Interfaces > Inter-Module Signals — The DV env drives the sleep_mode input (random initial value at reset, dropped to 0 on interrupt). (`hw/ip/aon_timer/dv/env/seq_lib/aon_timer_base_vseq.sv:213`, `hw/ip/aon_timer/dv/env/aon_timer_scoreboard.sv:980-989`)
- `f_0e434d9ee1a4` (spec not tested) Planned but not implemented: 5 requirement(s) in hw/ip/aon_timer/doc/interfaces.md > Interfaces … (4 sections) — The "planned only" claim is wrong for all five items. (`hw/ip/aon_timer/dv/aon_timer_sim_cfg.hjson:28-36`, `hw/dv/tools/dvsim/tests/tl_access_tests.hjson:20-23`)
- `f_2fc38343b1ac` (spec not tested) Register reset values: no suite covers every register — The generic dvsim csr_tests.hjson defines {name}_csr_hw_reset, which runs the common vseq with +csr_hw_reset over all registers. (`hw/dv/tools/dvsim/tests/csr_tests.hjson:21-24`, `hw/ip/hmac/dv/hmac_sim_cfg.hjson:30`)
- `f_e684b2415bb9` (spec not tested) Register access: no suite covers every register — The finding does not name its block. (`hw/dv/tools/dvsim/tests/csr_tests.hjson:29-43`, `hw/ip/hmac/dv/hmac_sim_cfg.hjson:30`)
- `f_fe240411fef7` (spec not tested) Register write enable locks: no suite covers every register — The testplan entry maps this item to the generic csr_rw and csr_aliasing tests, which run on every block through the shared CIP library. (`hw/dv/tools/dvsim/testplans/csr_testplan.hjson:99`)

Withdrawn items from the groups in section 5 (38):

- AON_TIMER.WAKEUP.WAKEUP_REQUEST: the AON_TIMER can wake up the core from sleep mode. — pwrmgr_sleep_all_wake_ups_impl.c configures the AON timer wakeup, sleeps, and checks the wakeup cause. (`sw/device/tests/pwrmgr_sleep_all_wake_ups_impl.c:272-273`)
- A watchdog bark interrupt is generated when the watchdog count reaches the threshold value. — Bark thresholds are randomized, with the count near the threshold, and the scoreboard checks the bark interrupt. (`hw/ip/aon_timer/dv/env/seq_lib/aon_timer_base_vseq.sv:100`)
- Program the thresholds in WKUP_THOLD_LO, WKUP_THOLD_HI, WDOG_BARK_THOLD and WDOG_BITE_THOLD. — The DV smoke sequence programs random WKUP_THOLD_LO and WKUP_THOLD_HI. (`hw/ip/aon_timer/dv/env/seq_lib/aon_timer_base_vseq.sv:192-196`)
- The wakeup threshold is a 64-bit value accessed through two 32-bit hi and lo registers. — DV writes random 64-bit thresholds split across the LO and HI registers. (`hw/ip/aon_timer/dv/env/seq_lib/aon_timer_base_vseq.sv:192-196`)
- Writing 0 to WKUP_CAUSE clears the wakeup request when the timer has caused a wakeup event. — DV writes 0 to WKUP_CAUSE. (`hw/ip/aon_timer/dv/env/aon_timer_scoreboard.sv:767-769`)
- The wakeup count and threshold are each 64-bit values accessed through two 32-bit registers. — The base vseq randomizes 64-bit wkup_thold and wkup_count and writes the LO/HI halves; the scoreboard checks the resulting register state and expiry. (`hw/ip/aon_timer/dv/env/seq_lib/aon_timer_base_vseq.sv:193-197`)

- X-value output assertions — ASSERT_KNOWN assertions in the RTL check outputs for X in every test. (`hw/ip/aon_timer/rtl/aon_timer.sv:290`)
- tl_intg_err — The aon_timer_tl_intg_err test is in the sim cfg through the imported tl_access_tests.hjson. (`hw/dv/tools/dvsim/tests/tl_access_tests.hjson:20-23`)
- pending_req_on_rst_cg — The shared tl_agent covergroup is sampled by the monitor. (`hw/dv/sv/tl_agent/tl_monitor.sv:103`)
- TLUL protocol assertions — tlul_assert is bound in aon_timer_bind.sv, which is in sim_tops. (`hw/ip/aon_timer/dv/sva/aon_timer_bind.sv:7-9`)
- SIM_SEC_CM_VERIFIED — The sec_cm testplan maps it to the aon_timer_tl_intg_err test. (`hw/ip/aon_timer/data/aon_timer_sec_cm_testplan.hjson:27-30`)
- csr_mem_rw_with_rand_reset — The shared csr_mem_rw_with_rand_reset test is imported by aon_timer and runs the reset sequence checks. (`hw/dv/tools/dvsim/tests/csr_tests.hjson:61-63`)
- csr_aliasing — The shared csr_aliasing test is imported by aon_timer. (`hw/dv/tools/dvsim/tests/csr_tests.hjson:45-50`)
- csr_bit_bash — The shared csr_bit_bash test is imported by the aon_timer sim cfg. (`hw/dv/tools/dvsim/tests/csr_tests.hjson:37-39`)
- sec_cm_bus_integrity — The shared aon_timer_tl_intg_err test exists and is imported by the aon_timer sim cfg. (`hw/dv/tools/dvsim/tests/tl_access_tests.hjson:20-23`)
- alert_test — The common aon_timer_alert_test is imported and uses +run_alert_test. (`hw/dv/tools/dvsim/tests/alert_test.hjson:14-18`)
- FPV_SEC_CM_VERIFIED — The checklist marks it N/A, so no test is expected. (`hw/ip/aon_timer/doc/checklist.md:223`)
- Software then writes WKUP_THOLD_HI with the upper 32 bits of the new threshold. — The THOLD_HI write is exercised by every smoke-style test through wkup_init, and the scoreboard checks the resulting interrupt behaviour. (`hw/ip/aon_timer/dv/env/seq_lib/aon_timer_base_vseq.sv:194-196`)
- nmi_wdog_timer_bark_o is a copy of intr_wdog_timer_bark_o. — The assertion wdog_bark_intr_is_nmi checks that the nmi output equals the bark output. (`hw/ip/aon_timer/dv/tb.sv:72`)
- Clearing WKUP_COUNT_HI and WKUP_COUNT_LO resets the wakeup timer count. — Count-register writes are modelled in the scoreboard and the expected interrupt timing is checked in DV. (`hw/ip/aon_timer/dv/env/aon_timer_scoreboard.sv:774-783`)
- Clearing the enable bit in WKUP_CTRL disables the wakeup timer. — The alternating enable/disable vseq stops the counter, and the scoreboard checks that no unexpected interrupt appears. (`hw/ip/aon_timer/dv/env/seq_lib/aon_timer_alternating_enable_on_off_vseq.sv:37`)
- Setting the enable bit in WKUP_CTRL re-enables the wakeup timer. — The scoreboard recalculates the expected wakeup time when the enable bit goes from 0 to 1, and the interrupt and wkup_req are checked against that. (`hw/ip/aon_timer/dv/env/aon_timer_scoreboard.sv:749-757`)
- The aon_timer module has a TL-UL bus device interface named tl. — The csr_rw test is imported in the aon_timer sim cfg. (`hw/ip/aon_timer/dv/aon_timer_sim_cfg.hjson:28-36`)
- lc_escalate_en is a 1-bit lc_ctrl_pkg::lc_tx unidirectional input (receiver). — The chip SW test exists and is registered. (`hw/top_earlgrey/dv/chip_sim_cfg.hjson:1250-1252`)
- tl is a 1-bit tlul_pkg::tl req_rsp interface on which this module is the responder. — The csr_rw test is imported in the aon_timer sim cfg. (`hw/ip/aon_timer/dv/aon_timer_sim_cfg.hjson:28-36`)
- The fatal_fault alert is triggered when a fatal TL-UL bus integrity fault is detected. — The tl_intg_err test is imported and run in the aon_timer regression. (`hw/ip/aon_timer/dv/aon_timer_sim_cfg.hjson:28-36`)

- The AON_TIMER.BUS.INTEGRITY countermeasure provides an end-to-end bus integrity scheme. — The tl_intg_err and sec_cm tests are imported and run in the aon_timer regression. (`hw/dv/tools/dvsim/tests/tl_access_tests.hjson:20-23`)
- AON_TIMER.WATCHDOG.DISABLE_BY_LC_CTRL: the watchdog is disabled when the life cycle controller escalates. — The chip-level test chip_sw_aon_timer_wdog_lc_escalate is implemented and in the regressions. (`sw/device/tests/aon_timer_wdog_lc_escalate_test.c:230-232`)
- sleep_mode is a 1-bit logic unidirectional input (receiver). — The input is driven by the testbench and the pause behaviour is checked through scoreboard interrupt-timing prediction. (`hw/ip/aon_timer/dv/env/seq_lib/aon_timer_base_vseq.sv:213`)
- regwen_csr_and_corresponding_lockable_csr — csr_rw and csr_aliasing are run from the imported common list. (`hw/dv/tools/dvsim/testplans/csr_testplan.hjson:102`)
- csr_hw_reset — The csr_hw_reset test is defined in the imported common list. (`hw/dv/tools/dvsim/tests/csr_tests.hjson:19-21`)
- tl_intg_err_cg_wrap — This is a covergroup. (`hw/dv/sv/cip_lib/cip_base_scoreboard.sv:60`)
- regwen_val_when_new_value_written_cg — This is a covergroup, implemented in dv_base_lockable_field_cov.sv and sampled by the common CSR tests. (`hw/dv/tools/dvsim/testplans/csr_testplan.hjson:102`)
- bit_toggle_cg_wrap — This is a covergroup, implemented in the shared bit_toggle_cg_wrap.sv. (`hw/dv/sv/dv_lib/bit_toggle_cg_wrap.sv:11`)
- INTR_TEST.wkup_timer_expired: the register is implemented outside the register block (true). — Covered by the generic intr_test, which checks the INTR_STATE and pin results of INTR_TEST writes. (`hw/ip/aon_timer/dv/aon_timer_sim_cfg.hjson:33`)
- INTR_TEST.wkup_timer_expired: hardware receives a write-qualifier pulse on software writes (true). — If hwqe did not pulse, writing INTR_TEST would not set INTR_STATE and the intr_test check would fail. (`hw/dv/sv/cip_lib/seq_lib/cip_base_vseq.sv:782-792`)
- INTR_TEST.wdog_timer_bark: the register is implemented outside the register block (true). — Same generic intr_test coverage applies to the wdog_timer_bark field. (`hw/ip/aon_timer/dv/aon_timer_sim_cfg.hjson:33`)
- INTR_TEST.wdog_timer_bark: hardware receives a write-qualifier pulse on software writes (true). — Same generic intr_test coverage applies to the wdog_timer_bark field. (`hw/dv/sv/cip_lib/seq_lib/cip_base_vseq.sv:782-792`)

Run record

Stage	Model
Spec extraction	claude-sonnet-5-5
Test extraction	claude-sonnet-5-5
Merge and link	claude-sonnet-5-5
Correlate	claude-sonnet-5-5
Analyze	claude-opus-5-5, claude-sonnet-5-5
Verify	claude-sonnet-5-5

This run: $8.17 in 1217 s (cached results reused where the inputs were unchanged). All LLM spend recorded for this workspace: $8.17.

Manifest `work/opentitan-aon-timer/manifest.yaml` (sha256 `18cbf293ec2c`); findings `work/opentitan-aon-timer/findings.yaml`; verifier transcripts in `work/opentitan-aon-timer/verify_runs/<finding id>/`.